

chap 07. Loop statement

Contents

01 Loops

02 while

03. do ~ while

04. for

05. break and continue

01. Loops

Loops

■ Loops

- Loops can execute a block of code as long as a specified condition is true.
- Loops are handy because they save time, reduce errors, and they make code more readable.

■ example

```
sum = 1+2+3+4+5+.....+100;
```

```
for( int i=1; i <= 100; i++ )
```

```
    sum = sum + i;
```

While Loop

■ while Statement

- The while loop repeats a block of code as long as a specified condition is true:
- Syntax

```
while (condition) {  
    // code block to be executed  
}
```

- example : print from 1 to 4

```
int i = 0;  
  
while (i < 5) {  
    printf("%d\n", i);  
    i++;  
}
```

Do/While Loop

■ Do/While Loop

- The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.
- Syntax

```
do {  
    // code block to be executed  
}  
while (condition);
```

■ Example

```
int i = 0;  
  
do {  
    printf("%d\n", i);  
    i++;  
}  
while (i < 5);
```

while vs do~whild (*attention*)

- The do/while loop always runs at least once, even if the condition is already false. This is different from a regular while loop, which would skip the loop entirely if the condition is false at the start.
- This behavior makes do/while useful when you want to ensure something happens at least once, like showing a message or asking for user input.

```
int i = 10;

while (i < 5) {
    printf("i is %d\n", i);
    i++;
}
```

no result

```
int i = 10;

do {
    printf("i is %d\n", i);
    i++;
} while (i < 5);
```

10

while loops

■ Challenge Code

- 1st : print even number from 1 to 10
- 2nd : Find the sum of the even numbers from 1 to 10.

```
int i = 1;

while (i <= 10) {
    if (i %2 == 0) {
        printf("%d ", i);
    }
    i++;
}
printf("\n");
```

```
int sum = 0;
int i=1;

while (i <= 10) {
    if (i %2 == 0) {
        // insert code
    }
    i++;
}
printf("sum of even =%d\n", sum);
```


for Statement

■ For Loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

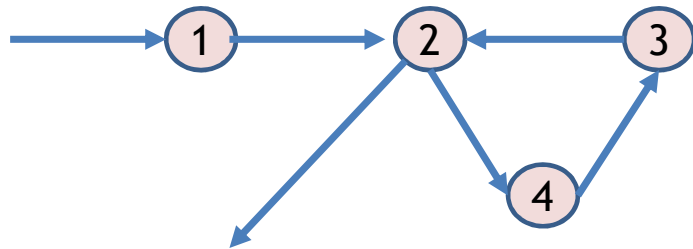
■ Syntax

	init		condition		iteration	
	1		2		3	
<code>for</code>	<code>(expression 1;</code>	<code>expression 2;</code>	<code>expression 3)</code>	<code>{</code>	<code>// code block to be executed</code>	<code>}</code>
				4	running code	

Expression 1 is executed (one time) before the execution of the code block.

Expression 2 defines the condition for executing the code block.

Expression 3 is executed (every time) after the code block has been executed.



for Statement

■ Example

- The example below will print the numbers 0 to 10

```
1 #include <stdio.h>
2
3 int main() {
4     int i;
5
6     for (i = 0; i <= 10; i++) {
7         printf("%d\n", i);
8     }
9
10    return 0;
11 }
```

This example prints even values between 0 and 10:

```
1 int i;
2
3 for (i = 0; i <= 10; i = i + 2) {
4     printf("%d\n", i);
5 }
```

for Statement

■ Nested Loops

- It is also possible to place a loop inside another loop. This is called a nested loop.
- The "inner loop" will be executed one time for each iteration of the "outer loop":

```
1 #include <stdio.h>
2
3 int main() {
4     int i, j;
5
6     // Outer loop
7     for (i = 1; i <= 2; ++i) {
8         printf("Outer: %d\n", i); // Executes 2 times
9
10        // Inner loop
11        for (j = 1; j <= 3; ++j) {
12            printf(" Inner: %d\n", j); // Executes 6 times
13            (2 * 3)
14        }
15    }
16    return 0;
17 }
```

for loops

- You can change the while loops to a for loops.

```
int i = 1;

while (i <= 10) {
    if (i %2 == 0) {
        printf("%d ", i);
    }
    i++;
}
printf("\n");
```

```
for ( int i =0; i <= 10; i++){
    if (i %2 == 0) {
        printf("%d ", i);
    }
}
printf("\n");
```

for statement

■ Code Challenge

- find the divisor(or factor)
- example :
 - All positive divisors of 12 are 1, 2, 3, 4, 6, and 12.
 - It is divisible by n
 - The remainder is 0 when divided by n.

02. Jump

Break and Continue

■ Break

- It was used to "jump out" of a switch statement.
- The break statement can also be used to jump out of a loop.
- This example stops the loop when i is equal to 4:

```
int i;  
  
for (i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    printf("%d\n", i);  
}
```

Break and Continue

■ Continue

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.
- This example skips the value of 4:

```
int i;  
  
for (i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    printf("%d\n", i);  
}
```


Thank You !